

Zen Coding

<HTML might be all well and good for a browser to understand and interpret, but it isn't really something that is designed for humans to write>

By Kshitij Sobti

At times, it can be painfully verbose to repeat the same old opening and closing tags for each element in HTML. For those who feel this pain, there is Zen Coding, an awesome shorthand way of writing HTML code. With Zen Coding, even the most complex HTML structures can be written using a single line of text.

Zen Coding does away with the need to enter opening and closing tags, and has shorthands for popular attributes such as the id and class. It takes away the verbosity of HTML by creating a syntax that lets you specify the important things in HTML tags, attributes, and hierarchy that makes writing code a breeze.

But what is Zen Coding? It isn't a new standard that replaces HTML; well not exactly. It is a shorthand way of writing HTML code that can be used in a large number of web development software. Many popular software either directly support it, or have plug-ins that add support for it. For example, Dreamweaver and WebStorm support it directly, while Notepad++, Vim, Emacs and many others have plug-ins for the same.

Of course, in the end it produces HTML, so you will need to be familiar with the basics of HTML to understand the rest of this article. To try out the examples we list here, use the browser-based Zen Coding-enabled editor available at <http://zen-coding.ru/codemirror/>. While this is actually a demo, and not an actual product, we recommend for the purpose of this article since it will work across all platforms. You can however find a plug-in for your favourite editor at the Zen Coding project page: <http://code.google.com/p/zen-coding/>.

The Zen way

One of the core features of Zen coding is the ability to expand on abbreviated representations of HTML code. Let's take a simple example of a div tag that has a class of "sidebar", you would need to write it as: `<div class="sidebar"></div>`

However the really important parts here are the div, the sidebar, and the relation between them — i.e. that sidebar is the class applied to div. With Zen coding, the idea is to write this in as simple terms as possible, and then use a particular hotkey to have the Zen Coding engine expand it to good old HTML. The example we just looked at could be represented in the syntax of Zen Coding as: `div.sidebar`

Now look at a somewhat more complex example. We have a div (with an id "div1") and a class (class1) applied to it. Here is how you would represent it: `div.class1#div1` OR `div#div1.class1`

Both of the above will expand to:
`<div id="div1" class="class1"></div>`

The order of id and class might vary though, based on which of the above you used; but in this case, the end result is the same.

If you're familiar with CSS, you will notice how this is similar to the concept of selectors in CSS. There too one can represent an id using "#" (bang / hash) and a class using a period (.). Its similarity

to CSS selectors only makes it easier to learn; if you know CSS, you know most of the shortcuts Zen Coding has to offer already.

How does this expand, you might wonder. That answer depends on the IDE you use and the shortcut it implements for this feature. If you're using the online editor we mentioned above, all you have to do is enter the abbreviated code and press tab. With the Notepad++ plug-in, you will have to press [Ctrl] + [Alt] + [Enter] to expand from Zen code to HTML. Now, want to go a little further and see what else Zen Coding is capable of?

Let's write a tag that has multiple classes, an id, and a number of attributes: `div#nav.class1.class2[title=navigation onclick=somefunc()]`

This will expand to: `<div id="nav" class="class1 class2" title="navigation" onclick="somefunc()"></div>`

Similarly, Zen Coding can represent the entire HTML structure.

Adding some structure

How better to start with structures, than to look at tables. In HTML, you could have a basic single column, single row table as follows:

```
<table>
<tbody>
<tr>
<td></td>
</tr>
</tbody>
</table>
```

In Zen Code this would be simply: `table>tbody>tr>td`

The > simply means that the tag on the left contains the tag on the right. There isn't much need to single-column, single row tables though, so how about we make a single row table with 5 columns: `table>tbody>tr>td*5`

This will happily expand to:

```
<table>
<tbody>
<tr>
<td></td> <td></td> <td></td> <td></td> <td></td>
</tr>
</tbody>
</table>
```

The *5 specifies that the tag is to be repeated 5 times. If you want to have 3 rows, and 5 columns, simply type: `table>tbody>tr*3>td*5`

You can go ahead and use any of the shorthands you have learned previously. For example, to add the class "class1" to the table, an id of data, and a class of "row" to each row, do the following: `table#data.class1>tbody>tr.row*3>td*5`

The result is what you'd expect, we won't show it here, but do try for yourself. Now, just a class of "row" might not cut it in some cases, what if you want a class that depends on the row number? Which is